

Benchmarking WebAssembly Performance in Real-World Applications and Libraries

ABSTRACT

WebAssembly has been increasingly adopted to execute a variety of applications and use cases on internet browsers, promising near-native and safe execution. It has increasingly been a subject of various performance and safety investigations. In this experience paper, we focus on evaluating the performance and behavior of WebAssembly applications representative of *real-world* uses. We quantify browser and hardware performance as well as investigate behavioral discrepancies with the equivalent natively-compiled applications and JavaScript-native libraries. Our applications are drawn from four diverse computational domains: computer vision, machine learning, cryptography, and bioinformatics. We investigate multi-threading and SIMD optimizations for computer vision applications, as well as the viability of computer vision across browsers with WebAssembly. We find that memory utilization and performance of machine learning tasks exhibit a strong dependence on browser and backend strategies. The performance of representative cryptographic algorithms is in line with underlying hardware specifications except for an instance of potential overhead using the Firefox browser. Finally, we benchmark sequence alignment on protein sequences of varying sizes and investigate discrepancies in the output compared to a CPU execution of the same application.

1 INTRODUCTION

WebAssembly (abbreviated Wasm or WASM) is a binary instruction format designed as a portable compilation target for web applications [58]. Since its first release in 2017 [21], WebAssembly has seen significant adoption, with over 95% of installed browsers supporting it as of April 2023 [18]. WebAssembly’s potential to achieve near-native performance in web applications has unveiled new possibilities.

Khodaya man kheili tanham, komakam taghir bedam hame chiz ro. Komakam kon taghir bedam sharayet ro. Komakam kon behtar besham, komakam kon taghir bedam sharayet ro. Komakam kon, faghat komakam kon. He is delusional, what will happen to me. Who will come and who will go. What will happen to me! What will happen next! God knows. Why people always misunderstand me!

For instance, utilizing WebAssembly to process sensitive data efficiently on the client-side instead of the server-side could alleviate some privacy concerns. However, there have been few attempts to benchmark the performance of WebAssembly [11, 29, 49, 59], and these efforts have been largely

confined to well-known benchmark suites, rather than real-world libraries or applications. This raises several questions, such as: Does porting commonly used libraries to WebAssembly for use in web applications degrade their performance? If so, by how much? Would these ported libraries still perform better than JavaScript-native libraries that offer comparable functionality? Is it possible to port complete native applications to WebAssembly and achieve comparable performance?

In this paper, we attempt to answer these questions by benchmarking the performance of WebAssembly ports of real-world applications and libraries. Our investigation focuses on three distinct fields: machine learning, cryptography, and bioinformatics. We selected these specific areas based on their diversity as well as their alignment with the intended use cases of WebAssembly [57], that includes scientific visualization and simulation, encryption, P2P applications and image recognition, among others. Within these applications, we focus our attention on Kalign [31, 32], OpenSSL [4], and Tensorflow.js [48] configured with WebAssembly backend.

In summary, this paper makes the following contributions:

- We make the first attempt to measure WebAssembly performance in four diverse computational domains: computer vision, machine learning, cryptography, and bioinformatics.
- We investigate behavioral discrepancies between WebAssembly and the equivalent natively-compiled applications and JavaScript-native libraries.
- We identify critical functions that contribute the most to the performance increase of ML applications when utilizing WebAssembly.
- We investigate multi-threading and SIMD optimizations for computer vision applications, as well as the viability of computer vision across browsers with WebAssembly.

The rest of this paper is organized as follows. Section 2 covers related work on the topic. In Section 3, we provide essential background information related to the applications and libraries we focus on. Section 4 describes the methods and tools used to benchmark the applications, and Section 6 presents and analyzes the results. We conclude the paper in Section 7.

2 RELATED WORK

To quantify the performance of WebAssembly, researchers have mainly attempted to compare the WebAssembly implementations of programs with either their native or JavaScript counterparts. Haas et al. [22] compared the performance of WebAssembly to asm.js and native code. Jangda et al. [29] investigated the slowdown of WebAssembly in comparison to native code and concluded that while optimizations could be made to improve the program size and run-time, some limitations arise from the design constraints of WebAssembly itself.

While WebAssembly is expected to be faster than JavaScript due to its low-level nature, research has shown that this is not always the case. Spies and Mock [49] demonstrated that, on average, WebAssembly code is smaller in size and executes faster compared to equivalent JavaScript code. Yan et al. [59] compared the performance of WebAssembly with equivalent JavaScript and noted the effects of compiler optimization techniques and JIT optimization on the results.

The performance of specific algorithms in WebAssembly has been studied. For instance, Sandhu et al. [44] investigated the performance of sparse matrix-vector multiplication in WebAssembly, while Herrera et al. [25] conducted measurements on the performance of numerical programs in WebAssembly.

There has also been growing interest in the use of WebAssembly for computer vision. The most popular libraries OpenCV and TensorFlow have official documentation support [1, 60] and have been tested in current research, verifying WebAssembly to be one of the optimal choices in the case of reduced disk space and computational power [36, 53]. There has also been growing literature discussing its potential uses and limitations in real-world applications [23, 36, 53, 61].

Sequence alignment algorithms are central in bioinformatics to understand evolutionary relationships between organisms. Online services for performing sequence alignments are ubiquitous; among the most notable are BLAST [5] and EMBL-EBI Clustal Omega [34, 45]. With such services it is necessary to upload user data to a server in order to perform tasks such as sequence alignment. By necessity, this allows client data, which may potentially be sensitive or unpublished, to be accessed and retained by the server. Executing a bioinformatics tool in WASM is a potential cross-platform solution for data-local processing. As a particular example of this solution, the Nextclade genomics tool developed to detect SARS-CoV-2 variants of concern includes a web interface with a WASM target of the program, allowing for completely client-side processing of sequence data in a research setting where such data may grow or update

rapidly [2]. Chromatic is another example of a genomics tool compiled to WASM and accessible as a web-based tool [15].

To the best of our knowledge, the only work that has benchmarked the WebAssembly performance of cryptographic primitives is by Yan et al. [59] that benchmarked modified CHStone [24] benchmark implementations of primitives such as AES, Blowfish and SHA. However, these results are not indicative of real-world applications, as most of them use APIs provided by standard cryptographic libraries, such as OpenSSL, to carry out cryptographic operations.

3 BACKGROUND

3.1 Cryptography

Some of the envisioned use cases of WebAssembly include cryptographic primitives such as encryption, as well as applications involving cryptography such as peer-to-peer applications, VPN and remote desktops [57]. Furthermore, recent research [41, 55, 56] demonstrates that cryptographic operations, such as those involving cryptographic hash functions, are widely used real-world applications of WebAssembly. As certain cryptographic operations can be CPU- and/or I/O-intensive, it is important to benchmark WebAssembly implementations of extensively deployed cryptography software and JavaScript native cryptography libraries to help identify potential areas of improvement of efficiency and user experience. In this paper, we focus our investigation on benchmarking OpenSSL [4], given its widespread use among websites and applications [38].

OpenSSL categorizes cryptographic primitives, such as SHA-256 digest or AES-128 encryption, as *algorithms* [40]. Based on their purpose, these algorithms are further grouped into *operations*, such as encryption, message digest and key agreement. OpenSSL supports the following cryptographic operations:

- *Generating/Verifying Message Digests*. Supported algorithms include MD5, SHA-2, SHA-3, BLAKE etc.
- *Symmetric Encryption/Decryption (including AEAD)*. Supported algorithms include DES, AES, RC2, RC4, ChaCha20 etc.
- *Key and Parameter Generation for Asymmetric Cryptography*. Supported algorithms include RSA, DSA, ECDSA etc.
- *Asymmetric Encryption/Decryption of Envelopes*. Supported algorithms include RSA + AES-128, RSA + AES-256 etc.
- *Generating/Verifying Signatures*. Supported algorithms include CMAC, HMAC etc.
- *Key Agreement*. Supported algorithms include DH and ECDH.

In Section 5.1, we discuss the various algorithms that are benchmarked.

3.2 Biology

In bioinformatics and computational biology, several algorithms have been developed to analyze biological sequence data [8]. The “tokens” of biological sequences can come in different forms, such as DNA nucleotides (genome) or the expressed amino acids of a protein (proteome) [15, 50]. In pairwise sequence alignment, one compares and optimally aligns the tokens of only two sequences. However, a more challenging task is to find a multiple sequence alignment (MSA): Given two or more biological sequences of possibly varying length, what is the “best” relationship between all of the sequences in terms of token substitutions (mutations), additions, and deletions? An example of this problem is illustrated in Figure 1. Quantifying the distance and similarities of genes and expressed protein sequences is critical for investigating evolutionary relationships in the tree of life. Organisms that are closely-related are more likely to express identical long segments of proteins, with few mutations or gaps. On the other hand, organisms that diverged much farther in the past are more likely to express mutations, which may be random or alter the chemical role of part of the protein in some way. For instance, the cat and zebrafish sequences in Figure 1 have mutations with entirely different chemical properties compared to the consensus amino acids, which are dominated by mammalian sequences.

Fast and resource efficient algorithms for multiple sequence alignment (MSA), in particular, have historically been challenging to implement. Finding an exact optimal MSA is a computational task that becomes exponentially expensive with the number of sequences. One of the earliest MSA algorithms, CLUSTAL, used a pairwise sequence heuristic to compute the MSA for 92 modestly-sized sequences within 25 minutes on hardware available in 1988 [26]. Various developments have improved the speed and accuracy of MSA and homology searching from datasets as computational resources increased. One notable example is a class of MSA algorithm based on hidden Markov models [13, 14].

A large number of bioinformatics tools have plain implementations in C and C++. Recently, targeting these tools to WebAssembly [33] has been a subject of both pedagogical and practical interest: Deploying a tool as part of a web application is helpful for learning an algorithm on the sample, small-scale data. Furthermore, running these tools on a web browser opens up more control over what data needs to be stored locally or fetched from a server. In addition, submitting the alignment job through a server [43] might have some safety disadvantages if an input includes sensitive data. In addition, there is always a response delay on the alignment we receive from a server.

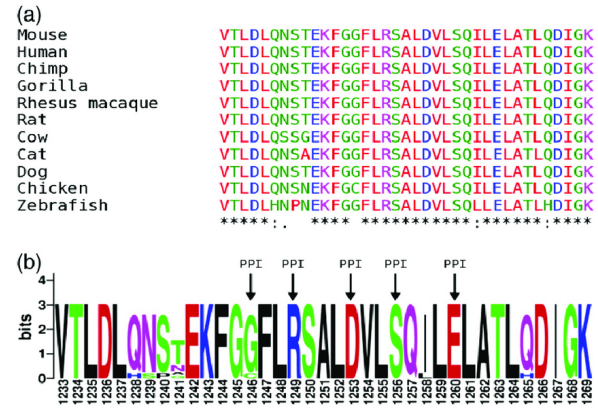


Figure 1: Example of a multiple sequence alignment of a protein sequence from a multi-exon gene, huntingtin (HTT). (a) This MSA spans 11 different animal species. Positions in the sequence that are 100% conserved in all these species are indicated with a (*). The coloring is a key to the different chemical properties of each amino acid in the sequence. (b) This visual histogram displays which positions are most conserved or vary with a certain amount of entropy [28].

3.3 Computer Vision Applications

Computer vision applications are a type of programs that perform analysis and processing to extract information from image data. Common tasks include object detection, tracking, classification, recognition, and segmentation. These programs are greatly varied in their use-cases with numerous examples such as AR, scanners, autonomous vehicles, and medical imaging.

The traditional methods to improve performance for intensive computer vision applications are to utilize native code features to have greater access to hardware and to upgrade and allocate more powerful hardware for the server where the computer vision application processing is hosted. With the ongoing hardware innovations that improve laptop and mobile devices performance, there has been a trend of migrating server-side processing to local machines. This trumps several of the disadvantages with server-side systems such as security and privacy concerns, network latency, and server costs.

3.4 Machine Learning

Machine learning (ML), with its various applications in diverse domains such as biology, finance, and industry [12, 37, 54], has become integral to many technologies. Most of these applications depend heavily on dedicated AI models, which often require too much computational power to run on edge devices. Thanks to the wide availability of WebGL in edge

devices, ubiquitous computing is becoming possible [16]. But this is still especially vital for those with low-end or no GPU. Consequently, web applications that require AI models typically partition models and other application components execution between the server and client sides. While it has a shortcoming [7] for large-scale cloud computing models: the unavoidable network delay that occurs during the exchange of requests and results between server and client presents a persistent challenge.

Network delays affect the user experience and potentially lead to lost revenue. In a cloud computing model, a delay in the network can result in the slow execution of services and applications. This is particularly vital [30] in applications like online gaming, video streaming, or other real-time services, where minor delays can cause significant disruptions.

The prospect of locally executed AI models [35] has been bolstered thanks to rapid advancements in CPU technology and continuous optimization. Google has pioneered web-based machine learning with TensorFlow.js [19], a comprehensive JavaScript library without server-side computation.

In addition, PaddlePaddle.js [17] represents another notable contribution to web-based machine learning. This potential was further augmented in 2017 when WebAssembly garnered comprehensive support from all major browsers. With WASM, some tasks with low-end or no GPU can be executed on the client side, mitigating network delay issues. In a WASM Guide book [47], the author cites a study by Google that found that WebAssembly can be up to 10 times faster than JavaScript for some tasks. Also, there has been a lot of development in optimizing large models designed for various fields to require less computational power for edge devices such as mobile for example reducing the number of computations and parameters by over 30 times from VGG-16 [46] to mobileNetv1 [27] for vision task's application.

4 METHODOLOGY

Applications ported to WASM can be measured by a variety of benchmarking metrics to ensure that (1) performance is not significantly lost relative to the native application, and (2) the application behaves, in general, as expected. Due to the design of web browsers and client-server communication protocols, these questions can have aspects that would not be a concern for CPU-compiled applications. Particularly complex applications may be affected by factors such as HTTP caching and can have non-trivial performance gap behavior. Furthermore, the use of certain compilation flags, pre-processor directives, and standard library functions may result in behavior that does not reproduce the CPU targeted application. In this section, we develop the benchmark rationale and detailed methodologies for each of the application

domains of interest, which are of varying internal and user-interfacing complexity. At a high level, our approach consists of four steps: (1) addressing output discrepancy, (2) compiling benchmarks to WebAssembly, (3) enabling HTTP caching, and (4) collecting metrics.

4.1 Addressing Output Discrepancy

To bring native applications to the web, it is important to verify that the execution of the application is identical between WASM and CPU targets. However, we observed output discrepancy issues that could cause compilation differences between WebAssembly and x86-64. In such cases, we preprocess the application to eliminate the issues that can potentially lead to output differences in different platforms. We will use a discrepancy observed in a biology application as an example to illustrate the problem and our solution.

An Output Discrepancy Example. We compiled the source code of the Kalign v3.3.1 package [31, 32] (the latest version available in the Biowasm CDN [20]) for two targets: x86-64 Linux CPU with GCC 8.3 and WASM with emcc 3.1. We observed a discrepancy in the output of both version (i.e. the aligned sequence) given the same input file which persisted even without optimization flags.

During an investigation of the Kalign source code, we initially discovered two issues potentially resulting in behavior differences in different platforms. First, even though the number of threads was just 1, the code nonetheless followed a different code path intended for parallel execution. This occurred because building to an x86-64 target discreetly enabled OpenMP whenever it was available. This was the case even when there was just one thread running in the background. Second, the serial code path was selected during the compilation process for a WebAssembly target. In contrast to the parallel code route, this code path used a random number generator (RNG) for the sampling stage instead of the parallel code path. A closer examination did not reveal this code path split or the RNG had an effect on the output discrepancy. However, it is worth noting that scientific applications are routinely developed with thread parallelism and RNG utilization. As a general remark, in porting to WASM it is an appropriate strategy to allow deterministic seeding of seed any RNGs and develop single-threaded reference implementations of critical algorithms.

Next, there were two more issues that affected the output of the results. This version of the Kalign source code also split the code path with a preprocessor directive when AVX2 instructions were available, which results in different implementations of routines for calculating the distance between sequences, particularly when targeting WebAssembly. Finally, the source code made frequent use of the `qsort` function from the C standard library, which is *not* guaranteed

to be a stable sorting function, and indeed is implemented differently in GCC (in glibc) and emscripten (in musl clib). This is critical because the sequences are at one point sorted by length, which is not a total order as different sequences can have the same length. In light of all of these issues, we decided to fork the Kalign repository and add features that would enable users to seed the RNG directly using the command line.

Our solution. In our fork, we addressed output discrepancy issues that could cause compilation differences between x86-64 and WebAssembly: (1) We allowed the RNG to be seeded through a command line option, (2) we disabled parallel execution with OpenMP where applicable, (3) we disabled the alternate code path with AVX2 instructions in the sequence distance calculation, and (4) we modified the ordering function passed to `qsort` to break sequence length ties by their index in the input file, so that the order becomes a total order and sorting is guaranteed to always yield the exact same result, regardless of sorting algorithm.

4.2 Compiling Benchmarks to WebAssembly

We used Emscripten to compile benchmark programs into WebAssembly and generate associated JavaScript code. Programs that benchmarked application-specific operations on files were compiled with the Emscripten flag `-preload-file` to package the file in the virtual file system for WebAssembly to access. Then, for each program, we hand-wrote minimal HTML files that invoked the corresponding WebAssembly module to perform the application-specific operation being benchmarked. The HTML files also contained code to measure the time taken to complete the WebAssembly module call using `performance.now()` method provided by JavaScript. The benchmark code is shown in Listing 1.

Listing 1: JavaScript Benchmark Code.

```
let sum = 0;
for (let i = 1; i <= NUM_TRIALS; i++) {
  // measure start time
  let start = performance.now();
  for (let j = 1; j <= NUM_TIMES; j++) {
    await func(...args); // run the function
  }
  // measure end time
  let end = performance.now();
  let time = end - start;

  // accumulate time taken for trials
  sum += time;
  console.log(`${libname} trial number: ${i}`);
  console.log(`\t${funcName} time: ${time}ms`);
}
```

```
console.log(`\tfinished ${funcName} ${NUM_TIMES}
times\n`);
}
// get mean time for trials
const average = sum / NUM_TRIALS;
```

For cryptography applications, we started by compiling OpenSSL 3.0.8 into WebAssembly using Emscripten [10]. We then wrote C programs that benchmarked AES-256 and SHA-256 algorithms. The AES-256 benchmark encrypted a message, decrypted it back and checked whether the result was equivalent to the original plaintext. The SHA-256 benchmark generated message digests and verified them. We wrote separate benchmark programs that performed cryptographic operations on files to better understand the impact of file size and file I/O overhead on OpenSSL’s performance in WebAssembly. All benchmark programs accessed OpenSSL through EVP API [39] calls to the `libcrypto` library.

4.3 Enabling HTTP Caching

In a browser with V8 engine, the default code caching scheme is implemented based on the cold, warm, and hot runs of the program [52]. In the cold run, the script is downloaded within the browser and passed to V8. The program script is then compiled in V8 for the first time, and the file is stored in the browser’s on-disk cache. When the same script is compiled the second time in V8, serialization of the compiled code takes place and metadata containing the serialized code is created and attached to the script. Next time the same script runs, V8 engine just deserializes the metadata passed from the browser and compilation is no longer needed. To enable WASM code caching, TurboFan mode is used in the first compilation. Then the cached code is retrieved and deserialized in V8 the second time a `.WASM` file is requested while skipping compilation [6].

In this study, TurboFan mode was used to enable optimizing compiler for both JavaScript and WebAssembly. TurboFan mode is on by compiler settings under `chrome://flags`. Specifically, WebAssembly Tiering and WebAssembly baseline compiler were enabled. To implement multiple loads of the same page, Puppeteer was used to simulate the web app actions. This study simulated one cold run and three hot runs of the same website.

To investigate the effect of the WebAssembly modules on the web app, response time of each WebAssembly module during the page loading was studied as the metric. For the response time measurement, start time was stamped when the network request for the WebAssembly module is sent. End time was measured when the response of the request is received.

Within the script, page loading is set up with `networkidle0` option. With this option, the page is determined loaded when

there are no more than 0 network connections for at least 500ms. This option was set to ensure that all resources on the page have loaded before proceeding to the next set of hot runs. Additionally, consecutive runs of the web app is loaded on a single page to simulate this caching experiment. For page loading, navigation timeout of 600000ms was set due to the resource constraint.

4.4 Collecting Metrics

We wrote a Python script using Selenium [3] library to load the webpage corresponding to each benchmark using Chrome, Firefox, Safari, and Edge browsers. The browsers were run in headless mode to minimize rendering overhead, and in ‘incognito’ or equivalent mode to minimize the effect of browser caching. For each method, input, and browser combination, a test was run multiple times, and the time taken to complete the WebAssembly module call was recorded for each run.

For biology applications, we used a repeated measures design to analyze the data, with the three types of methods for DNA sequence alignment. In addition, we used the same test to analyze the performance of multiple sequence alignment across the four browsers. The data were analyzed using repeated measures ANOVA, with $\alpha = 0.05$ set as the criterion for statistical significance.

Fundamental Operations in Machine Learning. To gain insight into the performance differences observed when transitioning from a no-WASM backend to a WASM backend in machine learning applications, a deeper investigation into the fundamental operations involved is necessary. In order to facilitate this investigation, we conducted a series of experiments with different browsers, loading strategies, backends, and target operations.

In each run of our experiments, we inserted two arrays of 10 million float32 numbers (a and b) into a testing function (Figure 2). This function was used to measure the execution time required to perform a targeted operation with a constant value c, along with the two input arrays. The results of these operations were stored as a' and b', respectively. We then evaluated the time required to execute the operation again, this time to compare the difference between the operation with a and b, compared to a' and b'. Finally, we monitored the time required to retrieve the final answer from the previous calculation.

Our experiments were designed to provide insight into the factors that may be contributing to the observed performance differences between no-WASM and WASM backends. By closely examining the execution time distribution of the target operation and its associated calculations, we aimed to use statistic techniques such as P-values to identify any

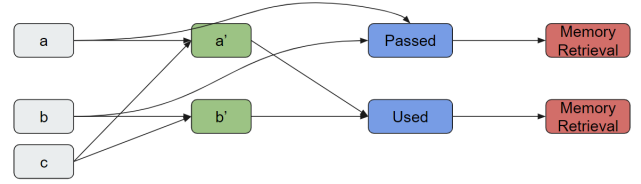


Figure 2: Fundamental Operation Investigation.

patterns or trends that could help explain the observed performance differences.

5 DATASETS AND EXPERIMENTAL SETUP

5.1 Datasets

Computer Vision. In order to measure the performance of browser computer vision applications with WebAssembly, we created benchmark performance tests on the de-facto computer vision library OpenCV. We first compiled OpenCV 4.7.0 into multiple configurations that can be run on a browser: (1) asm.js; (2) WASM; (3) WASM + MT (multi-threading); (4) WASM + SIMD (single instruction, multiple data); and (5) WASM + MT + SIMD.

We then perform 30 different commonly used image processing functions. The data we collected are the average of 10 trials, where for each trial we ran a function 100 times on Chrome, Firefox, Safari, and Microsoft Edge. To further test the feasibility of real-time processing, we apply these image processing and computer vision operations to video on all the same configurations and sample the FPS. At the time of this experiment, SIMD is not supported on Safari, so the results for SIMD related optimizations are not run on this browser. The code snippet below shows the method used to record the time for each function that was run in JavaScript.

An equivalent version of the code was used to run for just the images in opencv-python 4.7.0 for a comparison between the JavaScript and native runtimes.

Machine Learning Models. ML models are designed to perform a variety of tasks. In this study, we focused on natural language processing (NLP), as the benefits of these tasks can be easily noticed by a wide range of users. Specifically, we implemented a question-answering system using the MobileBERT [51] model for NLP tasks.

Cryptography. We focused our benchmarks on AES-256 symmetric encryption and SHA-256 message digest algorithms for several compelling reasons. Firstly, these algorithms represent some of the fundamental operations in cryptography, since symmetric encryption/decryption and message digest generation/verification serve as building blocks for various cryptographic operations. For instance, asymmetric encryption/decryption relies on symmetric encryption

internally, while cryptographic signatures (e.g., HMAC) rely on message digests. Moreover, these operations are often executed multiple times within a single session. In contrast, key and parameter generation for asymmetric cryptography is typically performed no more than once every several sessions. Similarly, key agreement usually takes place once per session. Lastly, these operations are performed on messages of various sizes ranging from a few bytes, such as transmitting a keystroke over SSH, to several gigabytes, such as secure file transfers. Each algorithm chosen was considered secure according to currently recommended standards [9].

Biology. We use the Kalign [31, 32] and Seq-align packages for protein and DNA sequence alignment, respectively, available in precompiled form in the Biowasm CDN [20]. To evaluate the performance of multiple sequence alignment, we employed the Kalign package along with a diverse set of ten protein sequence variants. For evaluating the performance of DNA sequence alignment, we utilized the Needleman-Wunsch and Smith-Waterman algorithms, implemented through the seq-align package. Kalign was also employed as the third method in this particular task, enabling a comparison of the three methods.

5.2 Experimental Setup

As part of our experimental setup, we used two common user devices to run the ML tests: a laptop with an i7-10870H processor and a desktop with an i5-10600K processor. These devices were chosen to provide a representative sample of the types of devices that are commonly used by end-users.

The laptop in our experiments featured an Intel Core i7-10870H processor with a base clock speed of 2.2 GHz and a maximum turbo frequency of 5.0 GHz. The desktop, on the other hand, featured an Intel Core i5-10600K processor with a base clock speed of 4.1 GHz and a maximum turbo frequency of 4.8 GHz.

To study the impact of machine specifications on the loading time for cryptography applications, the tests were independently run on two different PCs. PC-1 had an Intel Core i7-4770 CPU clocked at 3.4GHz along with 16GB of RAM, while PC-2 had an Intel Core i7-10875H CPU clocked at 2.3GHz along with 32GB of RAM. Both PCs ran OpenSUSE Tumbleweed as their operating system.

All experiments for the caching part of the study were run on a single MacBook Pro with specs as below: processor 2.7 GHz Intel Core i5 Memory 8 GB 1867 MHz DDR3 Graphics Intel Iris Graphics 6100 1536 MB macOS Mojave.

6 EVALUATION

To investigate the potential performance improvement of real-world applications using WebAssembly, we perform a

set of experiments to answer the following research questions:

- **RQ1 (Multithreading and SIMD):** What effect do multithreading and SIMD have on WASM performance?
- **RQ2 (Cross-Browser Performance):** How does the WASM performance differ among browsers?
- **RQ3 (Critical Functions):** What functions contributed the most to WASM performance improvement?
- **RQ4 (HTTP Caching):** How does HTTP caching affect the overall application performance?

6.1 RQ1: Multithreading and SIMD

We investigate multi-threading and SIMD optimizations for computer vision applications, as well as the viability of computer vision across browsers with WebAssembly.

asm.JS vs WASM. We find that on average, single-threaded WASM performs very close to asm.js. This is especially true with Chrome, Safari and Edge. On Chrome WASM performs slightly better than asm.js on most functions. On Safari and Edge, asm.js performed slightly better than WASM for most functions. However, Firefox asm.js performs significantly worse than the other browsers where we see 19/30 functions of at least 1 standard deviation slower performance. This attributes to great difference between Firefox’s asm.js and WASM versions. These times demonstrate a clear difference between optimization strategies and implementation of JavaScript engines across different browsers.

WASM vs WASM MT vs WASM SIMD. Firefox performs well and seems to outperform the other browsers on most of the function tests when WASM + SIMD + MT is applied. For all browsers, WASM + SIMD on almost all functions result in either no change or improved performance. This behavior can be attributed to the fact that some algorithms may stay the same, while others that have SIMD applied always results in a performance increase. The SIMD configuration seems to impact the performance on algorithms that require the use of OpenCV kernel sizes, where a single instruction can be run across multiple data points for effective optimization e.g. Median Blur. In our tests we also noticed that when Haar Cascades and Gaussian Blur are run on Firefox with SIMD enabled, no output is produced. This may indicate a difference in implementation of WASM + SIMD on the browser. MT seems to perform the worst on Safari compared to other browsers, even reducing the performance of functions such as Median Blur of up to 2000ms (20%). MT across browsers seem to have a greater disparity in performance. For algorithms such as Haar Cascades, we see improvements of up to 3x when MT is applied. For functions such as gaussian blur, we see an improvement with either MT or SIMD versions and an even greater performance boost when both are enabled. Upon applying MT + SIMD to the results, consistent high

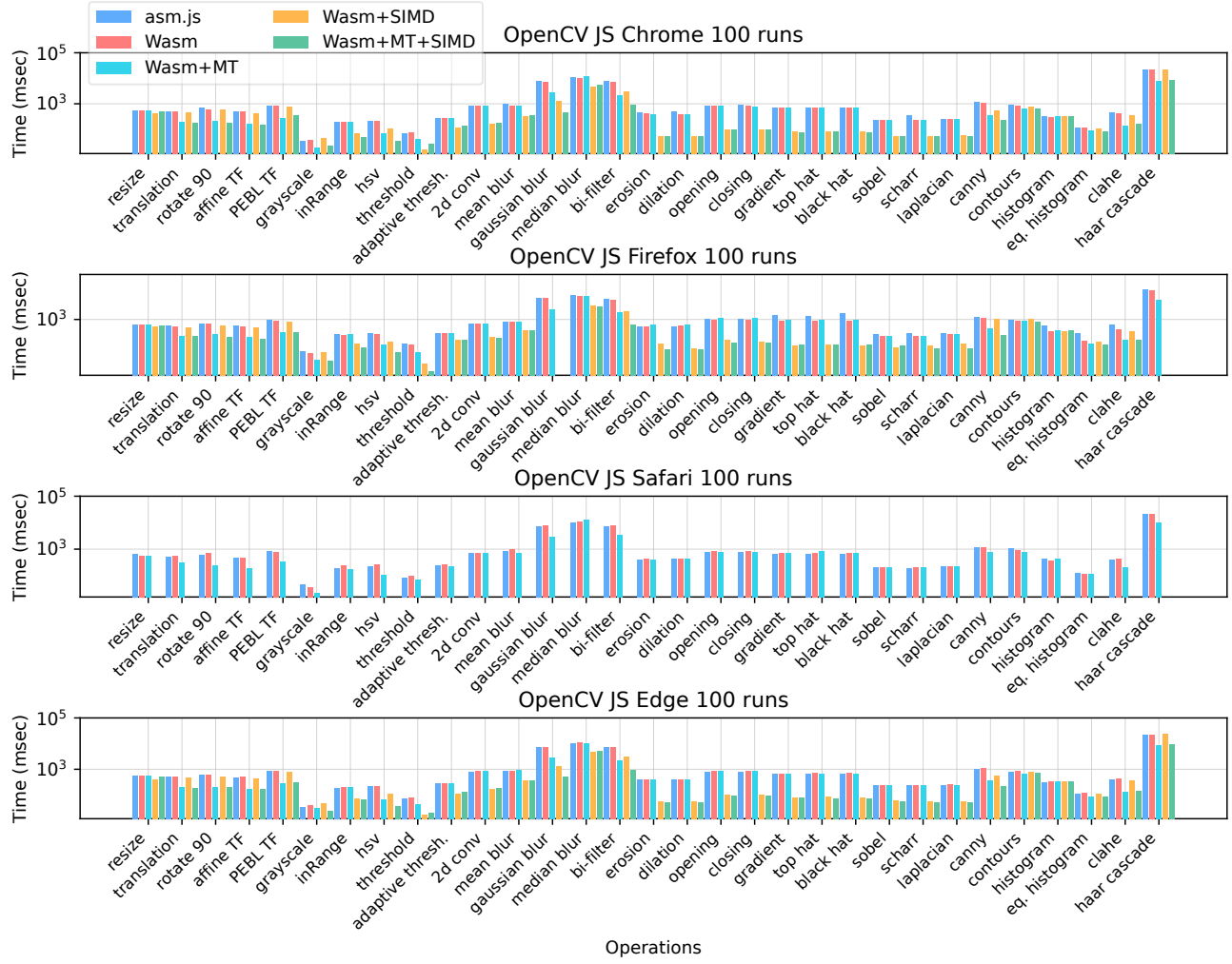


Figure 3: OpenCV JavaScript performance on Chrome, Firefox, Safari, and Edge.

performance can be achieved that approach native speeds, but performance is still lacking on certain functions.

Finding RQ1. The benchmark results demonstrate the viability of computer vision to perform well across browsers with WebAssembly. However, to be able to process every frame of a video at least 30fps, we need 100 runs of an image to run at around 3333ms or below. This condition is met for almost all functions when MT and SIMD are enabled except on Haar Cascades and Median Blur.

6.2 RQ2: Cross-Browser Performance

6.2.1 ML Models Performance. We ran 100 iterations of the MobileNet and MobileBERT models on an i7-10870H processor using different browsers and compared the results between the no-WASM and the WASM backends. More specifically, these two models are used to conduct image classification and question answerer respectively. This experiment can represent most of the tasks in vision and NLP field due to common base models, as most tasks in the real-world are also done by fine-tuning or transfer-learning based on these pre-trained models, thus they maintain a similar model structure to this experiment.

The performance results for the MobileNet-based image classification are depicted in Fig. 5. These results show that the Chrome browser outperforms the other browsers in both

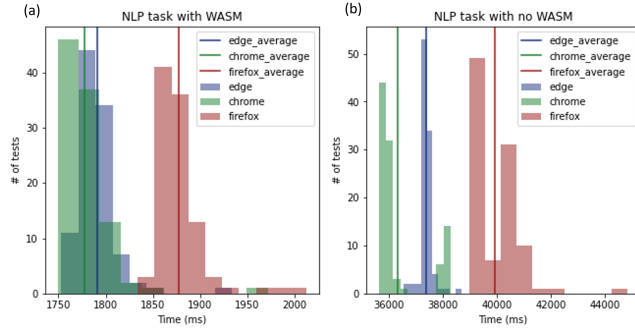


Figure 4: Execution time distribution of NLP task with WASM vs non-WASM.

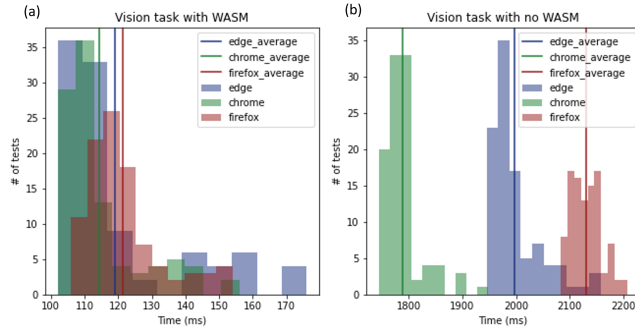


Figure 5: Execution time distribution of vision task with WASM vs non-WASM.

no-WASM and WASM environments. Additionally, the average run time for the vision task with WASM is 15.5 times faster than the no-WASM run time.

For the MobileBERT-based question answerer, we observed similar results. The results are shown in Fig. 4, indicating that the Chrome browser is again the fastest and Firefox is the slowest. Moreover, the average run time with WASM is around 20 times faster than the no-WASM run time.

Our findings suggest that, for both vision and NLP tasks, using the WASM backend can lead to significant performance improvements. Additionally, the choice of browser can also have a substantial impact on the overall performance of the ML model.

Finding RQ2-1. Performance evaluations of ML Models using the WebAssembly backend revealed significant improvements for both vision and NLP tasks, with the Chrome browser proving the fastest and Firefox the slowest in both WASM and non-WASM environments.

Finding RQ2-2. The performance gap between WASM and non-WASM backends for certain functions in vision tasks was striking, up to 21.5 times faster in WASM.

6.2.2 Cryptography Performance. To quantify cryptography application performance, we benchmark both string operations and file operations.

String Operations. As a first step, we benchmarked AES-256 and SHA-256 algorithms provided by OpenSSL-WebAssembly on the string “The quick brown fox jumps over the lazy dog.” The results of these benchmarks are presented in Table 1. Certain trends can be clearly observed from the table. Firstly, Chrome is the fastest of all three browsers, closely followed by Edge; whereas Firefox is the slowest across PCs and functionalities tested, with a higher mean latency as well as maximum latency. It can also be seen that PC 2 outperforms PC 1 across browsers and functionalities tested, which is not surprising since it sports a much newer processor as well as a higher amount of RAM. However, it is noteworthy that Firefox has a much better mean and maximum latency on PC 2 as compared to PC 1. This suggests that Firefox may be particularly slower on older/lower end machines.

File Operations. Next, we added the additional complexity of file I/O into the benchmarks. For this, we modified the benchmark programs to perform cryptographic operations on files pre-packaged by Emscripten at compile time. To investigate how file sizes affect the performance of OpenSSL-WebAssembly, we conducted tests using modified benchmarks with files of varying sizes: small (1 MB), medium (100 MB), and large (1 GB). Our selection of file sizes was based on real-world usage scenarios, as files commonly transmitted over the internet range from a few megabytes to several gigabytes.

The results of AES-256 and SHA-256 benchmarks are presented in Figure 12. The first takeaway is that files larger than a certain threshold result in very long execution times. For example, in the case of AES-256 encryption as run on Chrome, the 100x increase in file size from 1MB to 100MB resulted in a 56.19x increase in time, whereas a 10x increase in file size from 100MB to 1000MB resulted in a 9.94x increase in time. We hypothesize the reason for this trend as memory management issues associated with processing large files. We further investigate this hypothesis in Section 6.2.2.

For closer analysis, we focus on the benchmark results of AES-256 and SHA-256 algorithms on the medium (100MB) sized file. These results are presented in Table 1, which shows some interesting results. We observe that Firefox is no slower than Chrome or Edge on newer hardware when the computation time requirement is naturally high. In fact, on the newer PC, Firefox exhibits the best performance among the browsers in terms of maximum time taken. However, on older hardware, Firefox lags behind Chrome and Edge, consistent with findings from string-based cryptography benchmarks. But the slowdown is only 12.83% compared to the 103% difference in Table 1. This suggests that the significantly slower

Table 1: AES-256 and SHA-256 Benchmarks (String and File). Green numbers indicate the best performance among three browsers with the same group of PC. Red means the worse performance.

Operation	Browser	PC	String				File			
			mean (ms)	stdev (ms)	min (ms)	max (ms)	mean (ms)	stdev (ms)	min (ms)	max (ms)
AES-256	Firefox	Older	59	13	41	145	5,405	64	5,305	5,622
		Newer	34	2	26	41	3,439	72	3,321	3,675
	Chrome	Older	29	3	25	44	4,790	853	4,508	9,216
		Newer	21	2	18	26	3,406	132	3,234	3,835
	Edge	Older	31	3	27	41	4,630	105	4,518	5,093
		Newer	22	2	19	31	3,440	166	3,237	3,872
SHA-256	Firefox	Older	56	11	39	101	1,086	47	981	1,271
		Newer	32	2	24	39	732	25	678	797
	Chrome	Older	25	2	22	34	752	13	734	788
		Newer	19	2	16	27	586	9	567	608
	Edge	Older	27	2	24	34	769	17	747	833
		Newer	20	2	17	28	594	12	575	640

performance exhibited by Firefox while performing string-based cryptographic operations may be attributed to the startup time overhead of its JS/WASM engine, SpiderMonkey. This hypothesis is supported by the fact that Chrome and Edge utilize the same JS/WASM engine (V8) and exhibit nearly identical benchmark results. The further investigate this hypothesis in Section 6.2.2.

Comparison with JS-native library. The file benchmark results raise the question that whether the performance degradation is associated with large file sizes specific to OpenSSL-WASM or not. To answer this, we repeated the experiment using the browser-inbuilt JavaScript crypto.subtle library. The results are shown in Figure 12.

From Figure 12, it is clear that the performance degradation associated with large files is present while using the browser-native JS library as well. For example, in the case of AES-256 encryption, as run on Chrome, the 100x increase in file size from 1MB to 100MB resulted in a 46.91x increase in time, whereas a 10x increase in file size from 100MB to 1000MB resulted in a 12.06x increase in time. The fact that larger files seem to significantly degrade the performance of cryptographic algorithms irrespective of JS/WASM differences supports our hypothesis that this trend is caused by the memory management issues associated with processing large files.

From the results presented in Figure 12, it can be observed that Chrome and Edge have very similar statistics across all test categories. This may be due to the fact that both are Chromium-based browsers that use the same browser (Blink) and JavaScript (v8) engines.

Finally, we note that OpenSSL-WebAssembly performance of Firefox is consistently worse than Chrome and Edge when compared to native JavaScript cryptography library performance, as evident from Table 2. This is in line with our

Table 2: JavaScript native vs. WebAssembly OpenSSL of AES-256 and SHA-256 benchmarks. All data is in msec.

	AES-256		SHA-256	
	JS	Wasm	JS	Wasm
Firefox	16	180	22	87
Chrome	23	86	11	36
Edge	22	86	12	35

hypothesis that the weaker performance exhibited by Firefox may be attributed to its JS/WASM engine, SpiderMonkey. Thus, we conclude that the WebAssembly engine of SpiderMonkey needs improvement in order for Firefox to be on par with Chromium-based browsers.

Finding RQ2-3. Chrome and Edge, utilizing the same JS/WASM engine (V8), typically perform faster than Firefox when benchmarking AES-256 and SHA-256 algorithms using OpenSSL-WebAssembly on strings and files. Firefox performs comparably or even better than Chrome or Edge when benchmarking with medium-sized files (100 MB), which could be attributed to the startup time overhead of its JS/WASM engine, SpiderMonkey.

6.2.3 Biology Performance. Sequence alignment runtimes for Kalign are listed in Table 3, using sequences of varying size as measured by file size. The inputs included arbitrary unaligned protein families sourced from InterPro. This precursory profiling suggests that the execution of sequence alignment in Safari is slightly slower than in other browsers. The Safari browser is the only browser based on the Webkit engine. At the moment, it is not immediately clear why this engine performs notably slower with this application. The scaling of the runtimes for each browser with the size of the input in bytes is illustrated in Figure 6. Ideally, the

Table 3: Runtime of protein multiple sequence alignment using Kalign. All data is in msec.

Input size (B)	Chrome	Firefox	Safari	Edge
383	4	4	5	5
25,297	99	98	105	99
100,533	505	498	562	491
229,060	852	868	484	857
2,684,074	12,292	12,763	13,302	12,385
2,799,961	13,234	13,622	4,823	13,308
3,019,225	15,738	16,577	16,858	15,842
3,162,373	13,448	13,093	14,333	12,933
4,928,366	10,704	34,674	11,010	10,821
8,012,356	34,435	34,802	36,539	34,206

runtime of the alignment algorithm is linear with the input size, although some overhead inefficiency is to be expected. We note that Firefox scales the best out of all four browsers, although the lowest scaling by the Edge and Safari browsers is still about 0.90, and not dramatically lower than linear runtime.

Table 4 presents the results for DNA sequence alignment tasks. The results of the LCS task using Seq-align are available in Table 5. Note that this application, also built in the BioWasm repository, actually performs best in Safari, inversely to Kalign. The results of the repeated measures ANOVA revealed no significant difference between the four browsers regarding the runtime of the Kalign program ($F(3, 27) = 1.144, p > 0.05$). In addition, no significant difference was observed between the runtime of the three methods in the task of DNA sequence alignment ($F(2, 22) = 0.4242, p > 0.05$).

Finding RQ2-4. Execution of Kalign, a sequence alignment task in Safari is slightly slower than in other browsers, while execution of Seq-align, also a sequence alignment task performs best in Safari. However, repeated measures ANOVA revealed no significant difference in runtime between the four browsers for Kalign program ($p > 0.05$) and no significant difference in runtime for the three methods used in DNA sequence alignment tasks ($p > 0.05$).

6.3 RQ3: Critical Functions

In vision tasks, we assess the performance difference between WASM and non-WASM backends, revealing an average sevenfold disparity in favor of WASM. Figure 7 shows the functions analyzed on Chrome. These functions, fundamental in Convolutional Neural Networks (CNNs) and central to our study, greatly influence model performance and resource used. We selected recurrent benchmarks from CNN architectures for duration analysis and focused on those contributing the most to computational cost.

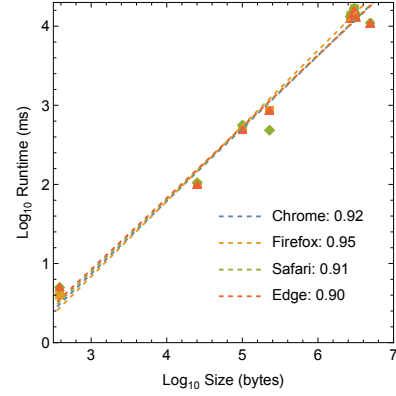


Figure 6: Runtime and scaling of Kalign in different web browsers with sequence alignment size. The best scaling is achieved by Firefox (0.95), while Safari (0.91) and Edge (0.90) have near-equally lowest scaling.

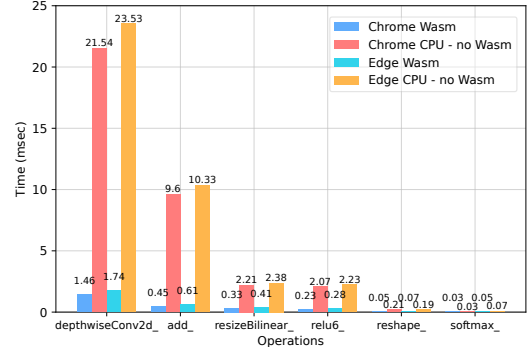


Figure 7: Average time cost for important function in vision task for different backends in Chrome and Edge.

Interestingly, we observed a comparable trend on the Edge browser. Functions such as fusedConv2d_, depthwidth2d_, and add_ demonstrated the most pronounced performance disparities between the two backends. The performance gaps in Edge for employing WASM versus no WASM backends for these functions were approximately 9.76, 13.52, and 16.93 times, respectively.

This result implies a significant impact on performance differences from functions beyond the monitored start function. We discovered that the fusedConv2d_ function which is computationally exhausting and the add_ function are two of the top most critical functions. After diving deeper into the implementation of the functions, the add_ function have some preconfirmation for example type checking and memory allocating, which turns out to be the factors that should be considered.

In CNNs, convolution primarily uses multiplication and addition, notably in the fusedConv2d_ function that accounts

Table 4: Runtime of DNA sequence alignment. All data except for input length is in msec.

Method	Input length	Chrome	Firefox	Safari	Edge
Smith-Waterman	100	212	7	6	313
	1,000	513	232	151	659
	10,000	4,196	2,781	2,059	4,780
Needleman-Wunsch	100	124	7	5	234
	1,000	179	34	39	265
	10,000	4,071	2,343	1,818	4,296
Kalign	100	1,193	41	30	1,596
	1,000	1,243	62	55	1,589
	10,000	3,053	1,627	538	3,551

Table 5: Runtime of Finding longest common substring in DNA sequence using Seq-align (in msec).

Input length	Chrome	Firefox	Safari	Edge
100	296	13	9	460
1,000	1,358	211	128	1,779
10,000	2,274	1,410	1,086	2,474

for a substantial part of CNN computational time [42]. Besides, the ReLu function in CNNs, relying on essential addition and subtraction, highlights CNN's dependence on arithmetic operations. Given this reliance, we next experiment with these operations using various backends and loading strategies to analyze the performance further.

Finding RQ3-1. The critical functions that have the most performance improvement when utilizing WASM are not necessarily computationally exhausting, but rather memory management and other factors' optimization matters significantly as well.

Fundamental Arithmetic Operations. Next, we investigate the performance difference in the fundamental arithmetic operations when transferring from a no-WASM backend to a WASM backend. We used both a desktop and a laptop to confirm the results, and we focused on four operations: addition, subtraction, multiplication, and division. However, we discovered a strange phenomenon during our tests: the first parameter sent into the function had a significant performance advantage over the second parameter when using the WASM backend, regardless of the operation being performed.

We initially assumed that this phenomenon could be attributed to the reference passing mechanism of JavaScript. However, after testing various input orders and execution orders, we found that the phenomenon still persisted, indicating that the location of the tensors was not the cause. We attempted to use P-value as a reference to determine the correlation between the execution time distribution and the input distribution, but we found that too much noise and

inconsistency resulted from observing wall time rather than CPU time. This could lead to biased conclusions if we relied solely on P-values.

Instead, we opted to use T-value directly, which revealed a clear pattern in the phenomenon. We used Figure 8 as an example to illustrate this pattern. In this figure, the x and y axes of each 4x4 boxes represent the execution time distribution for a', b', Passed, and the Used operations in Figure 2. The values 0 – 3, 4 – 7, 8 – 11, 12 – 15 on the axis represent four different types of arithmetic operations: add, sub, mul, and div, respectively. The color represents how much of an offset is the operation indicated in the y-axis compared to the x-axis. For example, when looking at (0,3), the 3 represents the "Used" operation in the addition arithmetic test, and the time it takes is significantly larger than the constant addition operation "a'", which is the 0. We can observe that the time taken to calculate a' is significantly less than the other three calculations in the WASM backend, but only in Chrome and Edge browsers. The phenomenon was not present in Firefox (left figure in Figure 9).

In summary, we found a strange phenomenon in our experiments where the first parameter sent into a function has a significant performance advantage over the second parameter when using the WASM backend, but only in browsers based on V-8 engine. We suspect that it is due to the different optimization strategies between the V-8 engine and spidermonkey for memory table allocation when calling WASM functions. There should be a lot more other factors, as what we observed is from the user's point of view (on browsers). Follow-up research is needed to understand what the factors are and what percentage is it contributing to the effect. In this paper, we noticed that using T-value directly allowed us to identify this pattern, and pointed out what properties might the user actual experience.

Finding RQ3-2. In Chrome and Edge, the first parameter sent into a basic arithmetic function has a significant performance advantage over the second parameter when using the WASM backend.

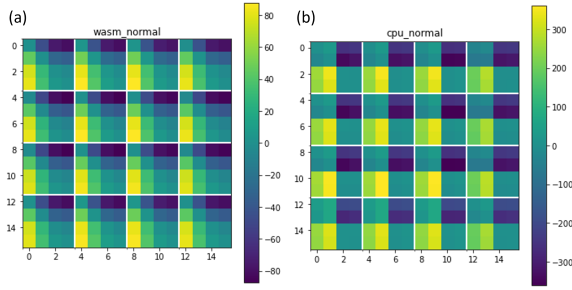


Figure 8: T-Value map comparison between WASM (a) and no-WASM (b) with Chrome and normal loading mode.

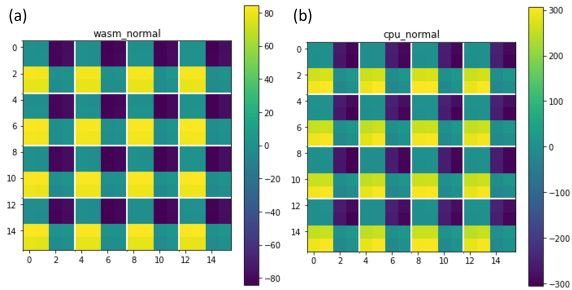


Figure 9: T-Value map comparison between WASM (a) and no-WASM (b) with Firefox and normal loading mode.

6.4 RQ4: HTTP Caching

In this experiment, the cold, warm, and hot runs of the program were indicated by the network activity on Javascript programs. Specifically, warm runs can be indicated when the scripts come from disk cache. Hot runs can be indicated when the scripts come from memory cache. These states were verified by checking the source of the scripts in the browser’s inspector tool under the network tab.

In order to study the HTTP caching effect on the web app performance across real-world applications, the test web apps were selected from a crawled list of 26 URLs that contain WebAssembly modules.

Figure 10 shows the WebAssembly response time of the cold run and the average of three hot runs in the unit of milliseconds. Figure 10 also shows the response time decreased from cold to hot runs. Overall, the optimized caching mechanism on the WebAssembly modules showed a substantial improvement in the performance of the web application. The response time of WebAssembly modules in each web application decreased significantly. Clearly, optimized caching of the WebAssembly modules improved the response by a factor of 81.15 on average compared to the cold run. This result aligns with the expectation considering the WebAssembly caching

Table 6: Average performance and speedup of WASM vs. without WASM. IMP means improvement. The unit of data in WASM and W/O WASM columns is msec.

Task	WASM	W/o WASM	IMP
Edge vision task	119.17	1,997.11	15.029x
Chrome vision task	114.41	1,788.58	15.537x
Firefox vision task	121.32	2,130.98	15.477x
Edge NLP task	1,791.04	37,402.71	18.728x
Chrome NLP task	1,777.59	36,337.35	20.316x
Firefox NLP task	1,877.68	39,916.36	18.731x

mechanism discussed previously. It is expected that the performance improvement with such an optimizing caching scheme will become even more significant when real-world applications involve greater network time.

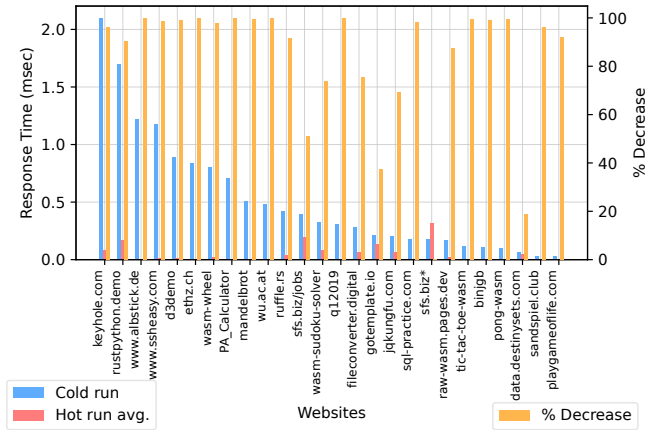


Figure 10: Percent Change in the Response Time of WebAssembly Modules during Cold Run and Hot Runs. The percent decrease for sfs.biz is -78.41 but set to 0 for a clearer view.

Finding RQ4. Optimized HTTP caching on the WASM modules led to substantial performance improvement, reducing response time by a factor of 81.15 on average.

7 CONCLUSION

In this paper, we make the first attempt to measure WebAssembly performance in four computational domains. We investigate behavioral discrepancies between WebAssembly and natively-compiled applications. We identify critical functions that contribute the most to the performance increase when utilizing WebAssembly. We also investigate multi-threading and SIMD optimizations for computer vision applications and the viability across browsers with WebAssembly.

CODE AVAILABILITY

The modified kalign branch is available at github.com/wasm-benchmarking/kalign-wasm. A minimal servable webpage with the WASM compiled executable is available at github.com/wasm-benchmarking/kalign-test. The benchmarking pipeline used for OpenSSL-WebAssembly can be found at github.com/wasm-benchmarking/openssl-wasm.

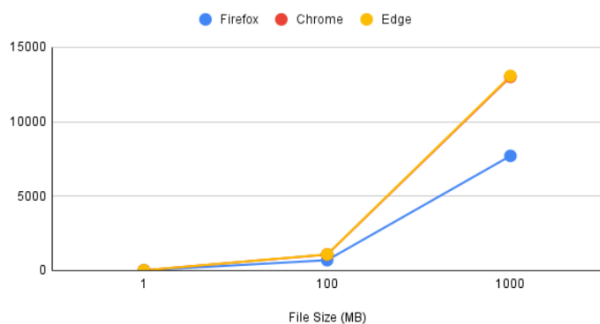
REFERENCES

- [1] [n. d.]. *OpenCV: Build OpenCV.js*. https://docs.opencv.org/4.7.0/d4/da1/tutorial_js_setup.html
- [2] Ivan Aksamentov, Cornelius Roemer, Emma B. Hodcroft, and Richard A. Neher. 2021. Nextclade: Clade Assignment, Mutation Calling and Quality Control for Viral Genomes. *Journal of open source software* 6, 67 (2021), 3773.
- [3] Rigzin Angmo and Monika Sharma. 2014. Selenium tool: A web based automation testing framework. *International Journal of Emerging Technologies in Computational and Applied Science, Chandigarh* (2014).
- [4] OpenSSL Project Authors. [n. d.]. OpenSSL. <https://www.openssl.org>. Last accessed on 2023-02-03.
- [5] Grzegorz M. Boratyn, Christiam Camacho, Peter S. Cooper, George Coulouris, Amelia Fong, Ning Ma, Thomas L. Madden, Wayne T. Matlen, Scott D. McGinnis, Yuri Merezuk, Yan Raytselis, Eric W. Sayers, Tao Tao, Jian Ye, and Irena Zaretskaya. 2013. BLAST: A More Efficient Report with Usability Improvements. *Nucleic Acids Research* 41, W1 (July 2013), W29–W33. <https://doi.org/10.1093/nar/gkt282>
- [6] Bill Budge. [n. d.]. *Code caching for WebAssembly developers*. <https://v8.dev/blog/wasm-code-caching>
- [7] Batyr Charyyev, Engin Arslan, and Mehmet Hadi Gunes. 2020. Latency comparison of cloud datacenters and edge servers. In *GLOBECOM 2020-2020 IEEE Global Communications Conference*. IEEE, 1–6.
- [8] Jacques Cohen. [n. d.]. Bioinformatics—an introduction for computer scientists. 36, 2 ([n. d.]), 122–158. <https://doi.org/10.1145/1031120.1031122>
- [9] Information Technology Laboratory Computer Security Division. [n. d.]. *Cryptographic Standards and Guidelines*. <https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines>
- [10] Emscripten Contributors. 2023. *Main - Emscripten 3.1.31-git (DEV) documentation*. *Main - Emscripten 3.1.31-git (dev) documentation*. <https://emscripten.org/>
- [11] João De Macedo, Rui Abreu, Rui Pereira, and João Saraiva. 2022. WebAssembly versus JavaScript: Energy and Runtime Performance. In *2022 International Conference on ICT for Sustainability (ICT4S)*. 24–34. <https://doi.org/10.1109/ICT4S55073.2022.00014>
- [12] Matthew F Dixon, Igor Halperin, and Paul Bilokon. 2020. *Machine learning in Finance*. Vol. 1170. Springer.
- [13] Sean R. Eddy. 2004. What Is a Hidden Markov Model? *Nature Biotechnology* 22, 10 (Oct. 2004), 1315–1316. <https://doi.org/10.1038/nbt1004-1315>
- [14] Sean R. Eddy. 2011. Accelerated Profile HMM Searches. *PLOS Computational Biology* 7, 10 (Oct. 2011), e1002195. <https://doi.org/10.1371/journal.pcbi.1002195>
- [15] Richard Finney and Daoud Meerzaman. [n. d.]. *Chromatic: Webassembly-based cancer genome viewer*. 17 ([n. d.]), 1176935118771972. Publisher: SAGE Publications Sage UK: London, England.
- [16] Carla Cristina Florez, Washington Quevedo, Francisco J. Galora, and Renato M. Toasa. 2021. Performance of WebGL standard for displaying 3D applications on mobile devices. In *2021 16th Iberian Conference on Information Systems and Technologies (CISTI)*. 1–6. <https://doi.org/10.23919/CISTI52073.2021.9476391>
- [17] Founder and Lead Developer-Bastian Allgeier. [n. d.]. *PaddlePaddle.js*. <https://developer.paddle.com/reference/5bbd4b69e0d8d-paddlejs-methods>
- [18] Fyrd. [n. d.]. WebAssembly | Can I use... Support tables for HTML5, CSS3, etc. <https://caniuse.com/wasm>
- [19] Charlie Gerard. 2021. *Practical machine learning in JavaScript: TensorFlow.js for web developers*. Springer.
- [20] Piotr Grzesik, Dariusz R Augustyn, Łukasz Wyciulik, and Dariusz Mrozek. [n. d.]. Serverless computing in omics data analysis and integration. 23, 1 ([n. d.]), bbab349. <https://doi.org/10.1093/bib/bbab349>
- [21] Andreas Haas, Andreas Rossberg, Derek L Schuff, Ben L Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 185–200.
- [22] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. Association for Computing Machinery, New York, NY, USA, 185–200. <https://doi.org/10.1145/3062341.3062363>
- [23] Raluca Maria Hampau, Maurits Kaptein, Robin van Emden, Thomas Rost, and Ivano Malavolta. 2022. An empirical study on the Performance and Energy Consumption of AI Containerization Strategies for Computer-Vision Tasks on the Edge. *The International Conference on Evaluation and Assessment in Software Engineering 2022* (06 2022). <https://doi.org/10.1145/3530019.3530025>
- [24] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. 2008. CHStone: A benchmark program suite for practical C-based high-level synthesis. In *2008 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1192–1195. <https://doi.org/10.1109/ISCAS.2008.4541637>
- [25] David Herrera, Hanfeng Chen, Erick Lavoie, and Laurie Hendren. [n. d.]. *WebAssembly and JavaScript Challenge: Numerical program performance using modern browser technologies and devices*. Technical Report.
- [26] Desmond G. Higgins and Paul M. Sharp. 1988. CLUSTAL: A Package for Performing Multiple Sequence Alignment on a Microcomputer. *Gene* 73, 1 (Dec. 1988), 237–244. [https://doi.org/10.1016/0378-1119\(88\)90330-7](https://doi.org/10.1016/0378-1119(88)90330-7)
- [27] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
- [28] Alis Hughes, Matthew Mort, Francesca A. Carlisle, and Lionel Jones. 2014. B04 Alternative Splicing In Htt. *Journal of Neurology, Neurosurgery & Psychiatry* 85 (Sept. 2014), A10–A10. <https://doi.org/10.1136/jnnp-2014-309032.32>
- [29] Abhinav Jangda, Bobby Powers, Emery D Berger, and Arjun Guha. 2019. Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code.. In *USENIX Annual Technical Conference*. 107–120.
- [30] Sujatha Krishnamoorthy, Zhangyu Wei, Yihuai Zhang, Hao Jin, and Hao Dong. 2020. A Study on Optimization of Network latency and Pocket loss Rate. *IOP Conference Series: Materials Science and Engineering* 937, 1 (sep 2020), 012054. <https://doi.org/10.1088/1757-899X/937/1/012054>
- [31] Timo Lassmann, Oliver Frings, and Erik L. L. Sonnhammer. 2009. Kalign2: High-Performance Multiple Alignment of Protein and Nucleotide Sequences Allowing External Features. *Nucleic Acids Research* 37, 3 (Feb. 2009), 858–865. <https://doi.org/10.1093/nar/gkn1006>

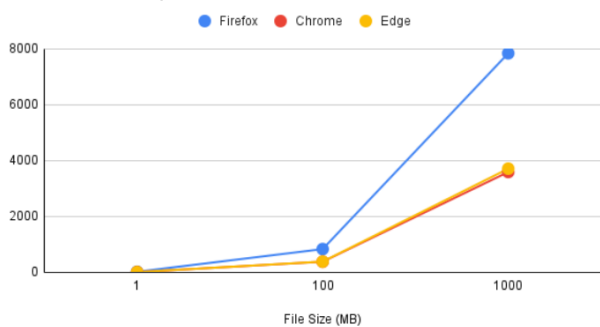
- [32] Timo Lassmann and Erik LL Sonhammer. 2005. Kalign – an Accurate and Fast Multiple Sequence Alignment Algorithm. *BMC Bioinformatics* 6 (Dec. 2005), 298. <https://doi.org/10.1186/1471-2105-6-298>
- [33] C. Lattner and V. Adve. [n. d.]. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* (2004-03). 75–86. <https://doi.org/10.1109/CGO.2004.1281665>
- [34] Fábio Madeira, Matt Pearce, Adrian R N Tivey, Prasad Basutkar, Joon Lee, Ossama Edbali, Nandana Madhusoodanan, Anton Kolesnikov, and Rodrigo Lopez. 2022. Search and Sequence Analysis Tools Services from EMBL-EBI in 2022. *Nucleic Acids Research* 50, W1 (July 2022), W276–W279. <https://doi.org/10.1093/nar/gkac240>
- [35] MG Sarwar Murshed, Christopher Murphy, Daqing Hou, Nazar Khan, Ganesh Ananthanarayanan, and Faraz Hussain. 2021. Machine learning at the network edge: A survey. *ACM Computing Surveys (CSUR)* 54, 8 (2021), 1–37.
- [36] Niko Mäkitalo, Tommi Mikkonen, Cesare Pautasso, Victor Bankowski, Paulius Daubaris, Risto Mikkola, and Oleg Beletski. 2021. WebAssembly Modules as Lightweight Containers for Liquid IoT Applications. *Lecture Notes in Computer Science* (2021), 328–336. https://doi.org/10.1007/978-3-030-74296-6_25
- [37] Diogo AC Narciso and FG Martins. 2020. Application of machine learning tools for energy efficiency in industry: A review. *Energy Reports* 6 (2020), 1181–1199.
- [38] Matus Nemec, Dusan Klinec, Petr Svenda, Peter Sekan, and Vashek Matyas. 2017. Measuring Popularity of Cryptographic Libraries in Internet-Wide Scans. In *Proceedings of the 33rd Annual Computer Security Applications Conference (Orlando, FL, USA) (ACSAC '17)*. Association for Computing Machinery, New York, NY, USA, 162–175. <https://doi.org/10.1145/3134600.3134612>
- [39] OpenSSL. [n. d.]. evp. <https://www.openssl.org/docs/man3.0/man7/evp.html>
- [40] OpenSSL. [n. d.]. libcrypto. <https://www.openssl.org/docs/man3.0/man7/crypto.html>
- [41] Jonathan Protzenko, Benjamin Beurdouche, Denis Merigoux, and Karthikeyan Bhargavan. 2019. Formally Verified Cryptographic Web Applications in WebAssembly. In *2019 IEEE Symposium on Security and Privacy (SP)*. 1256–1274. <https://doi.org/10.1109/SP.2019.00064> ISSN: 2375-1207.
- [42] Waseem Rawat and Zenghui Wang. 2017. Deep Convolutional Neural Networks for Image Classification: A Comprehensive Review. *Neural Computation* 29, 9 (2017), 2352–2449. https://doi.org/10.1162/neco_a_00990
- [43] Xavier Robert and Patrice Gouet. [n. d.]. Deciphering key features in protein structures with the new ENDscript server. 42 ([n. d.]), W320–W324. Issue W1. <https://doi.org/10.1093/nar/gku316>
- [44] Prabhjot Sandhu, David Herrera, and Laurie Hendren. 2018. Sparse matrices on the web: Characterizing the performance and optimal format selection of sparse matrix-vector multiplication in JavaScript and WebAssembly. In *Proceedings of the 15th International Conference on Managed Languages & Runtimes (ManLang '18)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3237009.3237020>
- [45] Fabian Sievers, Andreas Wilm, David Dineen, Toby J Gibson, Kevin Karplus, Weizhong Li, Rodrigo Lopez, Hamish McWilliam, Michael Remmert, Johannes Söding, Julie D Thompson, and Desmond G Higgins. 2011. Fast, Scalable Generation of High-Quality Protein Multiple Sequence Alignments Using Clustal Omega. *Molecular Systems Biology* 7, 1 (Jan. 2011), 539. <https://doi.org/10.1038/msb.2011.75>
- [46] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [47] Brian Sletten. [n. d.]. *Effect of WASM*. <https://www.oreilly.com/library/view/webassembly-the-definitive/9781492089834/>
- [48] Daniel Smilkov, Nikhil Thorat, Yannick Assogba, Charles Nicholson, Nick Kreeger, Ping Yu, Shanqing Cai, Eric Nielsen, David Soegel, Stan Bileschi, et al. 2019. Tensorflow.js: Machine learning for the web and beyond. *Proceedings of Machine Learning and Systems* 1 (2019), 309–321.
- [49] Benedikt Spies and Markus Mock. 2021. An Evaluation of WebAssembly in Non-Web Environments. In *2021 XLVII Latin American Computing Conference (CLEI)*. 1–10. <https://doi.org/10.1109/CLEI53233.2021.9640153>
- [50] Christopher S. Stipp, Tatiana V. Kolesnikova, and Martin E. Hemler. [n. d.]. Functional domains in tetraspanin proteins. 28, 2 ([n. d.]), 106–112. [https://doi.org/10.1016/S0968-0004\(02\)00014-2](https://doi.org/10.1016/S0968-0004(02)00014-2)
- [51] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. 2020. Mobilebert: a compact task-agnostic bert for resource-limited devices. *arXiv preprint arXiv:2004.02984* (2020).
- [52] Leszek Swirski. [n. d.]. *Code caching for JavaScript developers*. <https://v8.dev/blog/code-caching-for-devs>
- [53] Sajjad Taheri, Alexander Vedienbaum, Alexandru Nicolau, Ningxin Hu, and Mohammad R. Haghighat. 2018. OpenCV.js. *Proceedings of the 9th ACM Multimedia Systems Conference* (06 2018). <https://doi.org/10.1145/3204949.3208126>
- [54] Adi L Tarca, Vincent J Carey, Xue-wen Chen, Roberto Romero, and Sorin Drăghici. 2007. Machine learning and its applications to biology. *PLoS computational biology* 3, 6 (2007), e116.
- [55] Rodothea Myrsini Tsoupidi, Musard Balliu, and Benoit Baudry. 2021. Vivienne: Relational Verification of Cryptographic Implementations in WebAssembly. In *2021 IEEE Secure Development Conference (SecDev)*. 94–102. <https://doi.org/10.1109/SecDev51306.2021.00029>
- [56] Conrad Watt, John Renner, Natalie Popescu, Sunjay Cauligi, and Deian Stefan. 2019. CT-wasm: type-driven secure cryptography for the web ecosystem. *Proceedings of the ACM on Programming Languages* 3, POPL (Jan. 2019), 77:1–77:29. <https://doi.org/10.1145/3290390>
- [57] WebAssembly. [n. d.]. Use Cases - WebAssembly. <https://webassembly.org/docs/use-cases>, Last accessed on 2023-02-03.
- [58] WebAssembly. [n. d.]. WebAssembly. <https://webassembly.org/>
- [59] Yutian Yan, Tengfei Tu, Lijian Zhao, Yuchen Zhou, and Weihang Wang. 2021. Understanding the Performance of Webassembly Applications. In *Proceedings of the 21st ACM Internet Measurement Conference (Virtual Event) (IMC '21)*. Association for Computing Machinery, 533–549. <https://doi.org/10.1145/3487552.3487827>
- [60] Ann Yuan and Marat Dukhan. 2020. *Supercharging the TensorFlow.js WebAssembly backend with SIMD and multi-threading*. <https://blog.tensorflow.org/2020/09/supercharging-tensorflowjs-webassembly.html>
- [61] Aleksandra Zhuravleva. 2020. Progressive Web Camera Application using OpenCV WebAssembly module. *aaltodoc.aalto.fi* (08 2020). <https://aaltodoc.aalto.fi/handle/123456789/45984>

A THE RESULTS OF AES-256 AND SHA-256 BENCHMARKS

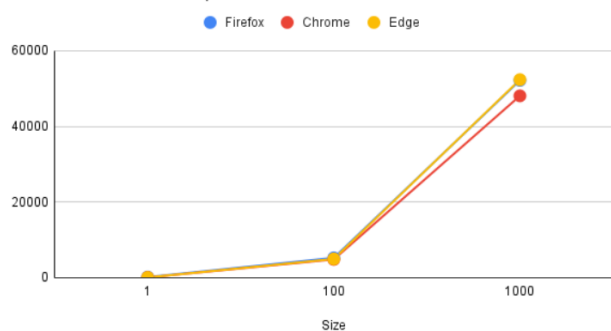
JS AES-256: Impact of File Size



JS SHA-256: Impact of File Size



WASM AES-256: Impact of File Size



WASM SHA-256: Impact of File Size

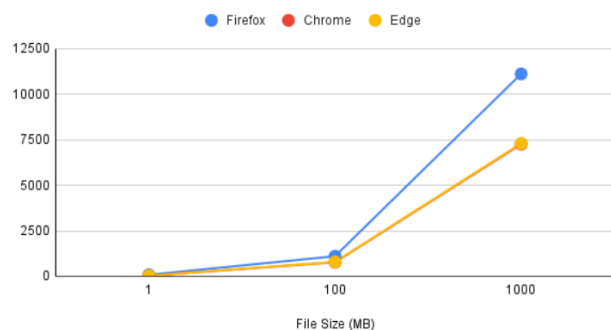


Figure 12: Impact of file size on browser native JavaScript crypto.subtle benchmarks and OpenSSL-WebAssembly benchmarks.